

Vault Encryption

How Vault files are protected, why the fingerprint prompt appears, and exactly what Rootaroo's own servers can and can't see.



Vault already uses genuine end-to-end encryption.

Every file is encrypted on the member's own device before it ever leaves it. Rootaroo's servers and cloud storage only ever hold scrambled bytes — they have no way to read a Vault file, with or without the fingerprint prompt. The details below show exactly how, step by step.

1

What "end-to-end encrypted" actually requires

A system only earns this label if it passes one test: could the people running the servers read your file if they wanted to — say, under a court order, after a breach, or just out of curiosity? For a true E2E system, the answer has to be **no**, because they never hold the key.

Vault passes that test. The file is scrambled (encrypted) before it leaves the phone, and the only key that can unscramble it never leaves the phone either. Rootaroo's database and file storage hold nothing but locked boxes and no working keys to any of them.

2

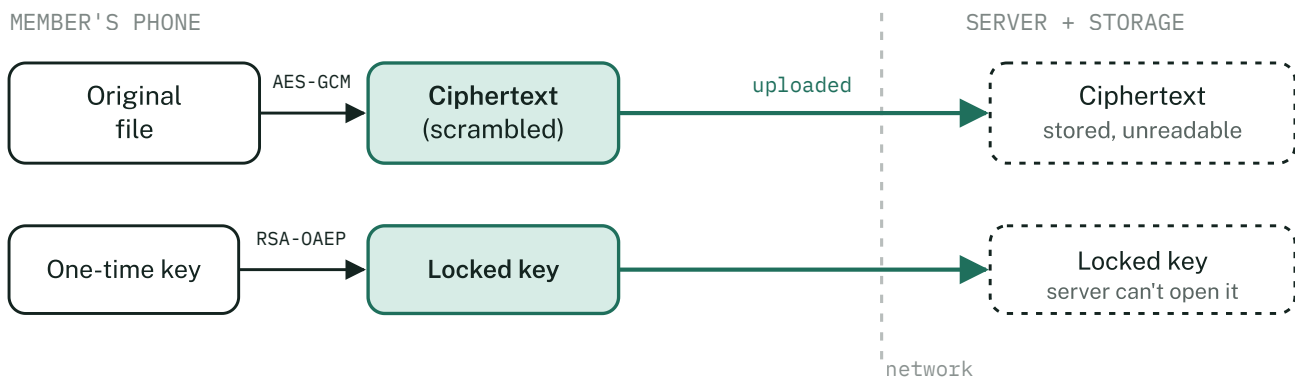
The mechanism, step by step

Two separate moments matter: uploading a file, and opening it again later. Each crosses the phone–server boundary exactly once, and in both directions, only ciphertext crosses.

Uploading a file

When a member adds a document to Vault:

1. The file's raw bytes are scrambled on the phone using a freshly generated, one-time key — **AES-256-GCM**, a standard used for things like disk encryption and secure messaging.
2. That one-time key is itself locked with the member's own personal public key — **RSA-OAEP, 4096-bit** — so only their matching private key can ever unlock it again.
3. The scrambled file and the locked key are uploaded. Nothing readable ever reaches the network.



Everything left of the dashed line happens on the member's own phone. Only the already-scrambled file and its already-locked key ever cross to the server — never the original file, never a usable key.

Opening a file later

The same steps run in reverse, entirely on the phone that opens the file:

1. The scrambled file and locked key are downloaded from storage — still just scrambled bytes at this point.
2. The phone asks the member to confirm it's really them, via **fingerprint or Face ID**, before it will release their private key from the phone's secure hardware. This step is covered in full in the next section.
3. The private key unlocks the one-time AES key (RSA-OAEP), which then unscrambles the file (AES-GCM) — entirely in the phone's working memory.
4. The unscrambled file is shown on screen. It is never written to the phone's disk or sent anywhere else.

SERVER + STORAGE

Ciphertext
+ locked key

downloaded

MEMBER'S PHONE

Fingerprint /
Face ID check

releases key

Phone's secure chip

unwrap key → decrypt → view (in memory)

The fingerprint check happens after the download, entirely inside the phone's own secure chip. It decides whether the phone will release the already-stored private key — the server is never involved in this step and never sees the result.

3 Why the fingerprint prompt doesn't weaken the encryption

This is the exact question that started this review, so it's worth answering directly: the fingerprint scan is **not** the thing protecting the file. The encryption above is. The fingerprint protects one specific thing — the member's private key — which already lives only on their own phone, inside hardware built for exactly this.

THINK OF IT LIKE A SAFE

The file itself is what's locked in the safe — that's the encryption, and it's real whether or not anyone ever scans a fingerprint. The fingerprint is just how the member unlocks their own key to their own safe. It's not a replacement for the lock; it's how one specific, legitimate person gets to use the key that was always theirs.

Concretely, on both iPhone and Android, a member's private key is stored inside the phone's dedicated security hardware (Apple calls this the **Secure Enclave**; Android calls it **StrongBox / TEE**) with a flag that tells that hardware: *never hand this key to any app — not even Rootaroo's own — without a fresh biometric check first*. That check happens entirely inside the chip. The app never receives or handles the actual fingerprint or face data, and there's no software path around it — the key simply isn't released otherwise.

One honest caveat: the fingerprint gate protects *the device*, not a password only the member knows. If someone already has a member's unlocked phone, they could pass that same fingerprint check the way they could unlock anything else on the phone. That's a device-security question, separate from whether the encryption itself is real — and the encryption is.

4 What Rootaroo's servers can see, at every step

A plain accounting of every moment a Vault file touches Rootaroo's infrastructure:

MOMENT	WHAT THE SERVER/STORAGE RECEIVES	READABLE BY ROOTAROO?
File is uploaded	Scrambled bytes only (AES-256-GCM ciphertext)	No
File's one-time key is uploaded	That key, itself locked with the member's public key (RSA-OAEP)	No
File sits in storage	Ciphertext at rest — same as above	No
Member opens the file	A time-limited download link is issued; the file itself flows straight to the member's phone	No
Member's private key is used	Never touches the server at all — it never leaves the phone's secure hardware	No

There is no step in this list where a readable file or a usable key passes through Rootaroo's servers. That's the actual definition of end-to-end encrypted, and it's true regardless of how a member authenticates on their own device.

5 Vault is private per member, by design

Vault is intentionally each household member's own private space — not a shared family folder. A file a member uploads is locked with only their personal key, so it's only ever visible to them, on their own devices. Other household members do not see it in their own Vault, and there's no shared "everyone's key" that could open it. This matches the intended design and is not a gap to close.

6 Known limitations, stated plainly

No security design is without tradeoffs. These are the real ones here, rather than a claim of perfection:

FIRST-USE TRUST WINDOW

The very first time a member's key is created, the phone has no prior record to check it against — it trusts the key it just generated ("trust on first use"). After that first moment, the phone will notice if that key ever unexpectedly changes. The gap is theoretical and narrow, but it exists, and there's no separate out-of-band check today to close it entirely.

OPTIONAL RECOVERY TRADES AWAY SOME GUARANTEE

At setup, a member can choose a passphrase-backed recovery option so losing their phone doesn't mean losing their Vault. Choosing that convenience means a backup of their key (protected by that passphrase) is held so it can be recovered — a deliberate, opt-in tradeoff against the alternative below, not a hidden weakening.

"NO RECOVERY" MODE MEANS EXACTLY THAT

A member can instead choose zero-knowledge mode with no backup at all. That's the strongest option, but it's irreversible in one direction: lose the phone, lose the Vault contents, permanently. This is by design, not a bug to report.

7 Spec sheet

For the record, the exact primitives in use:

FILE ENCRYPTION

AES-256-GCM

KEY WRAPPING

RSA-OAEP, 4096-bit

PASSPHRASE BACKUP KDF

PBKDF2-SHA256, 800,000 rounds

PRIVATE KEY STORAGE

Hardware keychain, biometric-gated

PLAINTEXT ON DISK

Never (in-memory only)

MITM KEY-SUBSTITUTION CHECK

Trust-on-first-use pinning

► Where this lives in the codebase (for engineering reference)

